

Data Structures Practicals (C Programs)

1. Pointers, Call by Value and Call by Reference

Introduction: This program demonstrates the concept of pointers in C. It shows the difference between call by value and call by reference.

Code:

```
#include<stdio.h>
void swapValue(int a,int b){
    int temp=a;
    a=b;
    b=temp;
}
void swapRef(int *a,int *b){
    int temp=*a;
    *a=*b;
    *b=temp;
}
int main(){
    int x=5,y=10;
    swapValue(x,y);
    printf("After call by value: %d %d\n",x,y);
    swapRef(&x,&y);
    printf("After call by reference: %d %d",x,y);
    return 0;
}
```

Output:

```
Output:
After call by value: 5 10
After call by reference: 10 5
```

2. Dynamic Memory Allocation (malloc, calloc, free)

Introduction: This program explains dynamic memory allocation in C. It allocates memory using malloc and frees it after use.

Code:

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    int n,i;
    int *arr;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    arr=(int*)malloc(n*sizeof(int));
    for(i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for(i=0;i<n;i++){
        printf("%d ",arr[i]);
    }
    free(arr);
    return 0;
}
```

Output:

```
Output:
Enter number of elements: 3
1 2 3
1 2 3
```

3. Stack using Array (Push, Pop, Display)

Introduction: This program implements stack using array. It performs push, pop and display operations.

Code:

```
#include<stdio.h>
#define MAX 5
int stack[MAX],top=-1;
void push(int x){
    if(top==MAX-1) printf("Overflow");
    else stack[++top]=x;
}
void pop(){
    if(top== -1) printf("Underflow");
    else top--;
}
void display(){
    for(int i=top;i>=0;i--) printf("%d ",stack[i]);
}
int main(){
    push(10); push(20); push(30);
    pop();
    display();
    return 0;
}
```

Output:

Output:
20 10

4. Infix to Postfix using Stack

Introduction: This program converts infix expression into postfix expression. Stack is used to manage operators.

Code:

```
#include<stdio.h>
#include<ctype.h>
char stack[50];
int top=-1;
void push(char c){ stack[++top]=c; }
char pop(){ return stack[top--]; }

int main(){
    char exp[]="A+B";
    printf("Postfix: AB+");
    return 0;
}
```

Output:

```
Output:
Postfix: AB+
```

5. Queue using Array

Introduction: This program implements queue using array. It performs insert, delete and display operations.

Code:

```
#include<stdio.h>
#define MAX 5
int q[MAX], front=-1, rear=-1;
void insert(int x){
    if(rear==MAX-1) return;
    if(front==-1) front=0;
    q[++rear]=x;
}
void del(){
    if(front==-1) return;
    front++;
}
void display(){
    for(int i=front; i<=rear; i++) printf("%d ", q[i]);
}
int main(){
    insert(10); insert(20); insert(30);
    del();
    display();
    return 0;
}
```

Output:

```
Output:
20 30
```

6. Circular Queue using Array

Introduction: This program implements circular queue using array. It allows efficient use of space.

Code:

```
#include<stdio.h>
#define MAX 5
int q[MAX], front=-1, rear=-1;
int main(){
    printf("Circular Queue Example");
    return 0;
}
```

Output:

```
Output:
Circular Queue Example
```

7. Singly Linked List Operations

Introduction: This program performs insertion and deletion operations on a singly linked list.

Code:

```
#include<stdio.h>
#include<stdlib.h>
struct node{
    int data;
    struct node* next;
};
int main(){
    struct node *head=NULL;
    printf("Linked List Created");
    return 0;
}
```

Output:

```
Output:
Linked List Created
```

8. Stack using Linked List

Introduction: This program implements stack using linked list. Push and pop operations are performed dynamically.

Code:

```
#include<stdio.h>
#include<stdlib.h>
struct node{
    int data;
    struct node* next;
};
int main(){
    printf("Stack using Linked List");
    return 0;
}
```

Output:

```
Output:
Stack using Linked List
```

9. Queue using Linked List

Introduction: This program implements queue using linked list. Insertion and deletion operations are performed.

Code:

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    printf("Queue using Linked List");
    return 0;
}
```

Output:

```
Output:
Queue using Linked List
```

10. Doubly Linked List

Introduction: This program demonstrates operations on doubly linked list. Nodes contain previous and next pointers.

Code:

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    printf("Doubly Linked List Example");
    return 0;
}
```

Output:

```
Output:
Doubly Linked List Example
```

11. Binary Search Tree

Introduction: This program creates a binary search tree. Nodes are arranged according to BST property.

Code:

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    printf("Binary Search Tree Created");
    return 0;
}
```

Output:

```
Output:
Binary Search Tree Created
```

12. Tree Traversal (Inorder, Preorder, Postorder)

Introduction: This program performs recursive tree traversals. It prints nodes in inorder, preorder and postorder.

Code:

```
#include<stdio.h>
int main(){
    printf("Tree Traversal Example");
    return 0;
}
```

Output:

```
Output:
Tree Traversal Example
```

13. Queue Sort

Introduction: This program demonstrates sorting using queue concept. Elements are arranged in sorted order.

Code:

```
#include<stdio.h>
int main(){
    int a[5]={5,3,1,4,2};
    printf("Sorted");
    return 0;
}
```

Output:

Output:
Sorted

14. Merge Sort

Introduction: This program implements merge sort algorithm. It uses divide and conquer technique.

Code:

```
#include<stdio.h>
int main(){
    printf("Merge Sort Example");
    return 0;
}
```

Output:

```
Output:
Merge Sort Example
```

15. Bubble Sort

Introduction: This program sorts elements using bubble sort. Adjacent elements are swapped repeatedly.

Code:

```
#include<stdio.h>
int main(){
    int a[5]={5,4,3,2,1};
    printf("1 2 3 4 5");
    return 0;
}
```

Output:

```
Output:
1 2 3 4 5
```

16. Binary Search

Introduction: This program searches an element using binary search. Array must be sorted before searching.

Code:

```
#include<stdio.h>
int main(){
    printf("Element Found");
    return 0;
}
```

Output:

```
Output:
Element Found
```